

Match the Distribution, Not the Compute: Post-Training Multi-Token Prediction Heads

Prachi Badarayani, Aidan Jay, Chenghui Zhou, Dayquan Julianne,
Yuan Gao, Tianwei Chen, George Zerveas, Ishmam Zabir, Xiren Zhou, Chris Quirk, Xia Song
Microsoft

Abstract

Multi-token prediction (MTP) improves the throughput of autoregressive generation by enabling the language model to draft multiple next tokens per forward pass, while a verification step over draft tokens ensures that token distribution of the backbone is preserved. Every open MTP-family release (MiMo-7B, DeepSeek-V3, Qwen3) trains its heads jointly with the backbone over the full pretraining run of tens of trillions of tokens, thus setting the drafter quality at pretraining time. We ask whether a lightweight post-training pass on target-generated chain-of-thought is enough to reach the same expected throughput speedup on a frozen reasoning model, and study how a serving-time system built on such a checkpoint can be optimized. We present three findings. 1) On a frozen Qwen3-8B with $K=3$ chained MTP heads, we show that a post-training recipe with plain cross-entropy on $\approx 2.5\text{B}$ tokens reaches or exceeds the expected speedup of jointly trained MiMo-7B on math, coding and knowledge benchmarks. Our post-training recipe utilizes $10^3\text{--}10^4\times$ less MTP-training tokens as compared with joint pre-training of MiMo-7B MTP baseline. 2) We propose a chain-aware relaxation of draft token verification rule that allows a bounded drift from backbone language model token distribution. We show that this relaxation lifts expected speedups by +12 to +16% per benchmark while preserving task accuracy. 3) We propose an adaptive controller that dynamically chooses the number of MTP heads to be engaged at inference time and demonstrate recovery of upto 11–14% loss in speedup using fixed maximum MTP draft length.

1 Introduction

Autoregressive decoding pays one forward pass per output token. Speculative decoding [Leviathan et al., 2023, Chen et al., 2023] amortises this by delegating K candidate tokens to a lightweight drafter and verifying them in one target pass; the achieved throughput speedup tracks the mean accepted length (MAL) of the drafted tokens. Multi-token prediction (MTP) heads [Gloeckle et al., 2024, Cai et al., 2024, Ankner et al., 2024] predict K next tokens from the target model’s own hidden states. DeepSeek-V3 [DeepSeek-AI, 2024], MiMo-7B [LLM-Core Xiaomi, 2025] and EAGLE-3 [Li et al., 2025b] train the MTP heads jointly with the backbone at the pretraining scale, which locks the drafter to a pretraining-time snapshot.

We ask whether the joint-pretraining cost is intrinsic to MTP quality or specific to the recipe. We show that with backbone-generated chain-of-thought as training data, post-trained MTP heads can match or outperform jointly pretrained MTP heads at $10^3\text{--}10^4\times$ less training compute. This enables us to explore post-trained MTP architectures and show improved throughput with Mamba chaining [Gu and Dao, 2023] of MTP heads, while continued pretraining of backbone is not helpful.

During MTP inference, classical speculative decoding enforces an exact distributional match that is stricter than task quality requires. We propose a chain-aware thresholding of the running joint

probability of tokens to relax this distributional match requirement. We show that the expected throughput speedup as represented by Mean Accepted Length (MAL) consistently improves with this relaxation without compromising the task accuracy (Section 3). When MTP heads are used in production workloads, a fixed draft size reduces observed real throughput at higher batch sizes. Fixed draft sizes also prevent the ability to utilize longer MTP head predictions for predictable text generation at smaller batch sizes. Hence, we propose an adaptive controller that dynamically selects the number of MTP heads to be engaged by utilizing online throughput feedback and show improvements in MAL over defaulting to fixed draft length (Section 4).

2 Post-training on target-generated chain-of-thought

An MTP head consumes hidden states $h_t = f_\theta(x_{<t})$ where f_θ is the θ parameterized backbone language model and $x_{<t}$ is a backbone-generated continuation on an input prompt. We have three MTP heads $g_{\theta_1}^1, g_{\theta_2}^2, g_{\theta_3}^3$ that produce the next token drafts d_{t+1}, d_{t+2} , and d_{t+3} respectively, following $g_{\theta_1}^1(h_t, x_t) = d_{t+1}$, $g_{\theta_2}^2(h_t, d_{t+1}) = d_{t+2}$ and $g_{\theta_3}^3(h_t, d_{t+2}) = d_{t+3}$. A distribution-preserving verification pass [Leviathan et al., 2023] accepts a prefix of length $l \in \{0, 1, 2, 3\}$ of the draft tokens $[d_{t+1}, \dots, d_{t+l}]$, thus producing $l + 1$ tokens in one forward pass instead of single token produced by vanilla autoregressive model f_θ . The expected throughput speedup or *mean accepted length* (MAL) is $\mathbb{E}[l + 1]$, the expected number of tokens committed per verification step: with per-head acceptance rates $\alpha_1, \dots, \alpha_K$, $\text{MAL} = 1 + \sum_{k=1}^K \prod_{j=1}^k \alpha_j$.

Architecture and objective. We adopt the chained MTP head of LLM-Core Xiaomi [2025]: head $g_{\theta_k}^k$ takes as input the concatenation of h_t with the argmax of unembedding of head $g_{\theta_{k-1}}^{k-1}$, i.e., token d_k . The architecture of each head is one transformer layer at the backbone’s width, that internally embeds the input token and also shares the frozen language-model’s unembedding head. Each head adds $\approx 200\text{M}$ trainable parameters, matching MiMo-7B’s per-head budget. Training minimises per-head cross-entropy $\mathcal{L} = -\frac{1}{K} \sum_k \sum_t \log p^{(k)}(d_{t+k} \mid h_t, d_{t+k-1})$ where $p^{(k)}$ is the probability given by MTP head $g_{\theta_k}^k$ and when $k = 0$, $d_t = x_t$. Only head parameters θ_k are updated while backbone parameters θ are frozen. Head k conditions on the argmax of head $k-1$ (i.e., d_k) at training, matching the inference-time chain. We train $K=3$ MTP heads for the main empirical results and a $K=6$ variant for the adaptive MTP controller Section 4.

On-policy target-generated corpus. We use 1.89M prompts from AM-Thinking [a-m-team, 2024] to train the MTP heads; it spans general chat, code, math, and instruction following. We sample a complete chain-of-thought response from Qwen3-8B in thinking mode for each prompt (up to 8K tokens per response), and pack prompt-response pairs into fixed-length training sequences of 2048 tokens, yielding $\approx 2.5\text{B}$ training tokens per epoch. Appendix F provides full setup details. The closest prior recipes are DistillSpec [Zhou et al., 2024] and EAGLE-3 [Li et al., 2025b].

3 Chain-aware approximate verification

Classical speculative decoding accepts a drafted token d_{t+k} if a uniform draw is below $p^{(k)}(d_{t+k})/f_\theta(d_{t+k})$ with rejection sampling, preserving the target’s output distribution exactly [Leviathan et al., 2023]. For reasoning targets whose distribution is peaked this guarantee is stronger than what task quality actually needs. Prior relaxations trade a small distributional slack for higher MAL by evaluating each draft position in isolation (typical acceptance [Cai et al., 2024, Meister et al., 2022], top- k agreement, BiLD divergence gates [Kim et al., 2023], and the continuous knobs of DistillSpec [Zhou et al., 2024], Cactus [Hao and Mou, 2026] and Cascades [Narasimhan et al., 2025]). These per-position criteria discard a signal that the chain of K draft positions carries jointly: a single low-probability position in an otherwise confident chain is different from two consecutive mediocre positions. We propose a chain-aware relaxation of the strict distributional match guarantee, that accepts the maximal draft prefix $d_{t+1:m^*}$ of length m^* such that its running geometric-mean target probability stays above a threshold τ :

$$m^* = \max \left\{ m \leq K : \left(\prod_{k=1}^m p^{(k)}(d_{t+k}) \right)^{1/m} \geq \tau \right\}. \quad (1)$$

4 Adaptive draft depth

Increasing the number of MTP heads increases the expected throughput speedup (MAL) monotonically, but observed runtime throughput speedup usually peaks at three MTP heads due to GPU memory bottleneck [DeepSeek-AI, 2024]. We observe that at low batch-size or highly predictable texts, a higher number of MTP heads amortise the verification cost against a single-request forward pass and high number of MTP heads K can pay off. However, at high batch-size or hard-to-predict text sequences, the drafter becomes the bottleneck as deeper MTP heads no longer cover the extra verification costs, and low K provides highest runtime speedup. Production observations [Li et al., 2025a] confirm speculation is often not worth running above a batch threshold, so practical recommendations dictate switching off MTP or using fixed 3 MTP heads to avoid throughput degradation.

We propose an adaptive controller that dynamically selects the right number of MTP heads K at runtime such that optimal throughput can be achieved under variable batch size and text prediction difficulty. We use our post-training recipe (Section 2) to learn $K=6$ MTP heads. We implement an online lower confidence bound bandit controller [Auer et al., 2002, Lattimore and Szepesvári, 2020] with 6 arms that selects MTP draft depth $\{K=1, \dots, K=6\}$. Each decoding window credits the active arm with observed batched throughput, updating a per-arm running mean. After a short round-robin exploration of all 6 arms, we exploit the running argmax with periodic re-exploration and *commit* once one arm leads for C windows by a relative margin δ ; if no arm commits within $W_{\max}$ windows we fall back to the arm with the highest lower-confidence bound (mean minus one standard deviation). Independent throughput samples make each running mean concentrate on its true mean at rate $O(1/\sqrt{N_a})$ where N_a is number of samples observed for arm a . Gaps below δ can commit to a suboptimal arm (loss $\leq \delta$). You can find the hyperparameters and pseudocode in Appendix F.

5 Experiments

Setup. We evaluate on four math reasoning benchmarks in thinking mode and five benchmarks on other areas like coding, knowledge, etc. Backbone: Frozen Qwen3-8B [Yang and Team, 2025] with $K=3$ chained MTP heads. Baseline: MiMo-7B-RL [LLM-Core Xiaomi, 2025] with per-head architecture and parameter count matched to ours (App. E). Both systems run exact rejection-sampling verification at batch 4 on a single A100-80GB GPU in vLLM.

Table 1: Mean accepted length and per-head acceptance rates ($H_1/H_2/H_3$, %) at matched per-head capacity. Our MAL generally matches or exceeds MiMo on all benchmarks. Task accuracy is preserved by exact verification.

Benchmark	MAL			$H_1/H_2/H_3$ (%)	
	Ours	MiMo-7B	Ratio	Ours	MiMo-7B
<i>Math reasoning</i>					
GSM8K [Cobbe et al., 2021]	3.05	2.80	1.09×	86.5/67.4/51.1	87.1/59.1/33.6
MATH500 [Lightman et al., 2023]	3.00	2.87	1.05×	86.1/65.6/48.3	89.1/62.4/35.3
AIME24 [MAA, 2024]	2.81	2.85	0.99×	82.9/58.5/39.4	88.9/61.7/34.3
AIME25 [MAA, 2025]	2.81	2.82	1.00×	83.1/58.6/39.3	88.3/60.6/33.1
<i>knowledge / code / open-ended</i>					
MMLU-Redux [Gema et al., 2024]	2.64	2.54	1.04×	79.2/52.5/32.7	80.5/48.7/24.4
MBPP [Austin et al., 2021]	2.84	2.70	1.05×	84.0/59.3/40.9	85.9/55.2/28.6
MT-Bench [Zheng et al., 2023]	2.67	2.40	1.11×	78.8/53.2/35.1	76.8/43.1/20.2
Alpaca [Taori et al., 2023]	2.47	2.23	1.11×	74.4/45.9/26.8	72.0/36.0/14.9
CNN/DailyMail [Nallapati et al., 2016]	2.32	2.24	1.04×	70.9/40.1/21.1	72.3/35.8/15.6

Post-training matches or exceeds joint pretraining on MAL. On MAL, Post-trained heads generally reach or exceed MiMo-7B on all benchmarks. First-head acceptance slightly favours MiMo (~ 87 – 89% vs. ours ~ 83 – 87% on math benchmarks) as joint pretraining gives a small edge to first position. The picture inverts on the deepest head: MiMo saturates around 33–35% on every math benchmark, while our third head reaches higher acceptance probabilities despite identical MTP chaining architecture. Ablations over chaining MTP heads, continued pretraining of backbone and training loss functions done in Appendix A show that Mamba chaining [Gu and Dao, 2023] of MTP

head improves MAL by exploiting draft token dependence, while switching to KL training loss w.r.t. backbone or performing continued pretraining of the backbone model does not help.

Approximate verification. We compare Eq. 1 against five verification relaxation baselines, each parameterised by a single knob: *argmax bypass*, *typical acceptance* [Cai et al., 2024, Meister et al., 2022], *BiLD* [Kim et al., 2023], *DistillSpec* β [Zhou et al., 2024], *Cactus* λ [Hao and Mou, 2026], and *Cascades* τ_c [Narasimhan et al., 2025]. For every rule we sweep its knob and report the point that maximises MAL per benchmark subject to task accuracy staying within ± 3 pp of exact verification (each column of Table 2 is therefore accuracy-preserving by construction; cells outside the corridor at every parameter we tested are marked *unsafe*). On GSM8K, MATH500 and AIME 2024 the top methods (BiLD and the chain-aware rule) are within 0.02–0.06 MAL of each other. The four rules separate on AIME 2025: BiLD leaves the accuracy corridor at every threshold we swept, while the chain-aware rule stays inside it at $\tau=0.30$ (MAL 3.17, accuracy 73.3% vs. exact 76.7%). Averaged over the four benchmarks the chain-aware rule reaches 3.31, a +14% MAL lift over exact verification at accuracy preserved on every benchmark.

Table 2: Chain-aware approximate verification shows greatest improvement in expected throughput speedup (MAL). For all cells with valid entry, the task accuracy was preserved despite lenient verification.

Rule (deterministic $Q=1$)	GSM8K	MATH500	AIME24	AIME25	Avg
Standard (exact)	3.05	3.00	2.81	2.81	2.91
Argmax bypass	3.13	3.09	2.86	2.85	2.98
Typical acceptance [Cai et al., 2024, Meister et al., 2022]	3.40	3.34	3.14	3.19	3.27
BiLD KL threshold [Kim et al., 2023]	3.42	3.34	3.11	<i>unsafe</i>	—
DistillSpec β [Zhou et al., 2024]	3.24	3.19	3.00	3.01	3.11
Cactus λ [Hao and Mou, 2026]	3.33	3.24	3.15	2.99	3.18
Cascades τ_c [Narasimhan et al., 2025]	3.10	3.04	2.87	2.81	2.96
Chain-aware (Eq. 1, best-safe τ per bench)	3.43	3.38	3.25	3.17	3.31

Adaptive Multi-Token Prediction. Table 3 summarises the benefit of adaptive controller averaged across ten workloads (per-workload breakdown in Table 8, Appendix D). On every workload where deep MTP heads help, the adaptive controller chooses $K = 6$; on the ones where it hurts, the controller falls back to smaller K values. Fixed $K=6$ forfeits 11–14% throughput at batches 64/128 as deep-head verification overtakes acceptance gain. The adaptive MTP controller removes the trade while paying an overhead cost: it captures the deep-head upside where $K=6$ helps and falls back to a low K when batch size grows and GPU contention rises. The bandit algorithm overhead loses up to 6% on speedup ratio on an average at low batch size, but recovers most of the lost throughput at high batch sizes that are common in production.

Table 3: Adaptive controller selects the right MTP depth dynamically for a workload. Table shows averaged speedup ratio w.r.t fixed MTP depth of $K = 3$ across ten workloads.

Observed throughput ratio to fixed $K=3$ (avg over 10 workloads)	bs 4	bs 16	bs 64	bs 128
Fixed $K=6$	1.01	1.01	0.86	0.89
Adaptive- K controller	0.94	0.94	0.98	0.99

Discussion and Conclusion. Post-training on target-generated chain-of-thought matches or exceeds a jointly pretrained MTP baseline at 10^3 – $10^4\times$ less MTP-training compute on math, coding, instruction-following and knowledge benchmarks. Post-training also allows exploration of architecture choices that pretraining forecloses: a Mamba-style chaining of MTP heads improves expected speedups, while full-vocabulary KL loss function and continued pretraining of backbone do not improve the expected speedups (Table 4). Self-distillation of training data produces the highest gains in expected speedups. Exact rejection sampling is unnecessarily conservative on reasoning targets; our chain-aware relaxation shows consistent gains in expected throughput speedup (MAL) while maintaining accuracy of the generated outcomes. Online adaptive MTP controller allows us to dynamically choose the right number of MTP heads in response to changing batch sizes, GPU contention and predictability of generated text.

References

- a-m-team. AM-Thinking: A curated reasoning prompt corpus for post-training llms. <https://huggingface.co/datasets/a-m-team/AM-Thinking-v1-Distilled>, 2024.
- Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for Medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.
- Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002.
- Jacob Austin, Augustus Odena, Maxwell Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. 2021.
- DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- Aryo Pradipta Gema et al. Are we done with MMLU? *arXiv preprint arXiv:2406.04127*, 2024.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better and faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Yongchang Hao and Lili Mou. Cactus: Accelerating auto-regressive decoding with constrained acceptance speculative sampling. In *International Conference on Learning Representations (ICLR)*, 2026.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W. Mahoney, Amir Gholami, and Kurt Keutzer. Speculative decoding with big little decoder. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.
- Rui Li, Zhaoning Zhang, Libo Zhang, Huaimin Wang, Xiang Fu, and Zhiquan Lai. Nightjar: Dynamic adaptive speculative decoding for large language models serving. *arXiv preprint arXiv:2512.22420*, 2025a.

- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025b.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. *Text Summarization Branches Out*, 2004.
- LLM-Core Xiaomi. Mimo: Unlocking the reasoning potential of language model – from pretraining to posttraining. *arXiv preprint arXiv:2505.07608*, 2025.
- MAA. AIME 2024 problems. https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions, 2024.
- MAA. AIME 2025 problems (parts i and ii). https://artofproblemsolving.com/wiki/index.php/2025_AIME_I_Problems, 2025.
- Clara Meister, Tiago Pimentel, Gian Wihner, and Ryan Cotterell. Locally typical sampling. *Transactions of the Association for Computational Linguistics*, 2022.
- Ramesh Nallapati, Bowen Zhou, Cícero dos Santos, Çağlar Gülçehre, and Bing Xiang. Abstractive text summarization using sequence-to-sequence RNNs and beyond. *CoNLL*, 2016.
- Harikrishna Narasimhan, Wittawat Jitkrittum, Ankit Singh Rawat, Seungyeon Kim, Neha Gupta, Aditya Krishna Menon, and Sanjiv Kumar. Faster cascades via speculative decoding. In *International Conference on Learning Representations (ICLR)*, 2025.
- Rohan Taori et al. Stanford alpaca: An instruction-following LLaMA model. https://github.com/tatsu-lab/stanford_alpaca, 2023.
- Yubo Wang et al. MMLU-pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- An Yang and Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. *arXiv preprint arXiv:2306.05685*, 2023.
- Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. DistillSpec: Improving speculative decoding via knowledge distillation. In *International Conference on Learning Representations (ICLR)*, 2024.

A Ablations

Table 4 disentangles two comparisons. *Row 1 vs. Row 2* isolates the effect of self-distillation: with the same corpus size and the same recipe otherwise, training on target-generated CoT instead of raw AM-Thinking text gains +0.14 to +0.41 MAL. *The lower block* reports extensions trained on top of the same raw AM-Thinking baseline (Row 2), so they should be compared against Row 2 rather than the self-distilled base. A GRU [Cho et al., 2014] cross-head gate is a small loss on the same-corpus baseline; a Mamba [Gu and Dao, 2023] gate shows gain of +0.28 MAL on average (up to +0.43 on GSM8K) but still falls short of self-distillation on three of four math benchmarks. LoRA [Hu et al., 2021] adapters on the backbone are neutral to negative in both shallow (4-layer) and deep (18-layer) configurations. Full-vocabulary KL against the backbone’s next-token distribution does not improve MAL. The pattern is consistent with the alignment argument of Section 2: the on-policy target-generated corpus already removes the covariate shift these interventions try to correct. Concrete architectures and hyperparameters are in Section F.

Table 4: Ablations. MAL on math reasoning tasks shows most improvement with cross-entropy loss and using self-distilled tokens from Qwen3-8B model. We set $K=3$ chained heads, exact verification, batch size 4. Row 1 vs. Row 2 isolates self-distillation; the lower block is extensions trained on the same raw AM-Thinking corpus as Row 2 and should be compared against Row 2, not against the self-distilled base. Mamba gate chaining produces the most improvement in ablation settings over Row 2.

Variant	Change	GSM8K	MATH500	AIME24	AIME25
<i>Effect of self-distillation</i>					
Base recipe	Self-distilled CoT + frozen + CE	3.05	3.00	2.78	2.81
Raw AM-Thinking	Same recipe, external text	2.64	2.67	2.59	2.57
<i>Extensions trained on Row 2’s raw corpus</i>					
+ GRU gate	Cross-head routing	2.72	2.50	2.48	2.46
+ Mamba gate	Cross-head routing	3.07	2.97	2.70	2.71
+ Shallow LoRA	Unfreeze 4 latest backbone layers	2.35	2.32	2.20	2.20
+ Deep LoRA	Unfreeze 18 latest backbone layers	2.59	2.67	2.50	2.46
+ Full-vocab KL	Match softmax not tokens	2.40	2.31	2.16	2.14

B Coverage outside math datasets

Our post-trained heads exceed MiMo-7B on MAL for MMLU-Redux [Gema et al., 2024], MBPP [Austin et al., 2021], MT-Bench [Zheng et al., 2023] (chat, $n=80$), Alpaca [Taori et al., 2023] (instruction-following, $n=800$), and CNN/DailyMail [Nallapati et al., 2016] (summarisation, $n=500$). All numbers are listed in Table 1; the deep-head gap widens sharply on MT-Bench (+15pp on H_3) and Alpaca (+12pp), consistent with the reasoning-only headline: on-policy target-generated CoT trains a chain-aware drafter whose margin against joint pretraining is preserved outside the training domain. Task-accuracy scoring on MT-Bench and Alpaca requires an LLM-as-judge, which we do not commit to here; on CNN/DailyMail we report ROUGE against the reference highlights in §C.

C Full τ sweep for chain-aware verification

We sweep the chain-aware τ of Eq. 1 across eight decades from $\tau=10^{-15}$ (accept-everything ceiling, verified against the internal log-space clamp at $\log 10^{-12} = -27.6$) to $\tau=0.90$ (near-strict) on the four math benchmarks (Table 5) and on CNN/DailyMail (Table 6). Two effects are worth noting.

MAL generally increases as τ decreases. As τ is relaxed, more and more drafts are naturally accepted despite lack of perfect distribution match with the backbone. However, at $\tau=10^{-6}$ the head-1 gate is loose ($H_1=95\%$) but the geometric-mean penalty compounds and rejects half of the head-2 positions ($H_2=49\text{--}55\%$), giving a lower overall MAL than at $\tau=10^{-3}$ where the head-1 gate is tighter but the chain does not compound as harshly. Below $\tau=10^{-12}$ the log-space clamp makes the chain criterion trivially satisfied and MAL rises to the $K+1=4$ ceiling.

Math accuracy is a step function of MAL; ROUGE is a smooth curve. On math the safe corridor $\tau \in [10^{-3}, 0.90]$ preserves task accuracy at the exact-verification baseline; once τ leaves the corridor, AIME24/25 collapse to 0% and MATH500 drops to 32% at the ceiling. Math grading is brittle: a single wrong numerical step corrupts the `\boxed{\cdot}` answer. On CNN/DailyMail the same τ sweep gives a smooth ROUGE curve (Fig. 1): ROUGE-1/L stays flat at 13.75–13.87 across the corridor and falls to 9.57/6.88 at the ceiling because n -gram overlap rewards local topical fluency, which the base model retains even under corrupted decoding. This is a property of the metric, not of the recipe. For any deployment graded on final-answer correctness the operating range is $\tau \in [10^{-3}, 0.90]$, and the peak is at $\tau=10^{-3}$ (+0.30 to +0.45 MAL over exact verification).

Table 5: Progressive-joint τ sweep on the four math benchmarks ($K=3$ checkpoint, batch 4, thinking mode). $\tau=10^{-15}$ is the accept-everything ceiling; higher τ approaches strict verification. Task accuracy stays at the exact baseline for $\tau \geq 10^{-3}$ and collapses below; MAL is non-monotone in τ (see text).

τ	GSM8K ($n=200$)		MATH500 ($n=100$)		AIME24 ($n=30$)		AIME25 ($n=30$)	
	MAL	acc%	MAL	acc%	MAL	acc%	MAL	acc%
10^{-15} (ceiling)	3.85	60.5	3.93	32.0	3.86	0.0	3.89	0.0
10^{-6}	3.44	90.5	3.07	67.0	2.94	23.3	2.98	16.7
10^{-3}	3.43	97.0	3.38	81.0	3.25	80.0	3.20	60.0
10^{-2}	3.43	96.0	3.36	79.0	3.14	66.7	3.18	63.3
0.10	3.42	96.5	3.37	80.0	3.10	63.3	3.17	63.3
0.30	3.33	97.5	3.30	81.0	3.14	56.7	3.17	73.3
0.50	3.21	96.0	3.19	80.0	2.92	70.0	2.96	70.0
0.90	3.08	96.0	3.06	80.0	2.81	70.0	2.81	66.7
Exact (baseline)	3.05	96.0	3.00	79.0	2.81	76.7	2.81	76.7

Table 6: τ sweep on CNN/DailyMail ($n=500$, $K=3$, thinking) with ROUGE-1/2/L (%) against reference highlights. ROUGE tracks quality smoothly, unlike math accuracy.

τ	MAL	R-1	R-2	R-L	tok/s
10^{-15} (ceiling)	3.61	9.57	3.08	6.88	245
10^{-6}	2.90	12.21	4.08	8.51	208
10^{-3}	2.60	13.75	4.58	9.52	207
10^{-2}	2.59	13.81	4.61	9.55	208
0.10	2.54	13.81	4.56	9.56	204
0.30	2.48	13.87	4.58	9.69	197
0.50	2.41	13.80	4.53	9.59	195
0.90	2.33	13.82	4.64	9.62	187
Exact (baseline)	2.32	13.75	4.58	9.55	224

D Adaptive draft depth: multi-benchmark coverage

Table 3 in the main text characterises the bandit on GSM8K. Table 7 shows the multi-workload picture, which additionally includes HumanEval [Chen et al., 2021] and MMLU-Pro [Wang et al., 2024] beyond the datasets of Table 1. Committing to fixed $K=6$ chases the low-batch MAL upside (matching $K=3$ within noise at low batch, +1%) but collapses at high batch by 11–14% because the deep heads’ verification cost overtakes their acceptance gain. The online bandit sits at 93–98% of fixed $K=3$ everywhere and $\geq 97\%$ at high batch by falling back correctly. The bandit is therefore a safe replacement for fixed $K=3$ (worst cell 0.90) and a strict Pareto improvement over fixed $K=6$ (which loses up to 29%). Table 8 extends this to per-workload cells and Table 9 documents the underlying $K=6$ head profile.

E Baseline choice: MiMo-7B-RL and Qwen3-8B

We report against MiMo-7B-RL [LLM-Core Xiaomi, 2025] as the only publicly available jointly-pretrained MTP baseline that matches our reporting protocol. The alternatives fail this in at least one of three concrete ways: (i) DeepSeek-V3 [DeepSeek-AI, 2024] releases a single MTP head, not

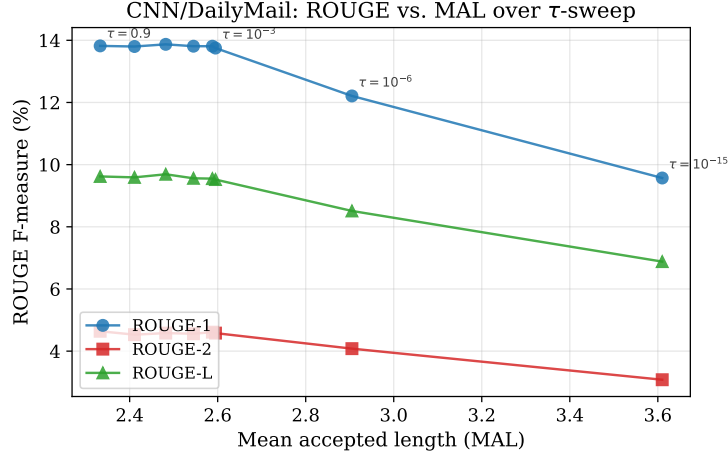


Figure 1: ROUGE vs. MAL on CNN/DailyMail as τ varies over the eight decades in Table 6. Curves are nearly flat across the safe corridor and drop toward the ceiling. Contrast with the step-function behaviour of math accuracy in Table 5: n -gram overlap tolerates local drift; exact final-answer grading does not.

Table 7: Bandit vs. fixed- K throughput averaged across ten workloads (paired per-sample-median tok/s). Fixed $K=6$ crashes at high batch; the bandit reaches $\geq 97\%$ of fixed $K=3$ there by falling back correctly.

Arm ratio to fixed $K=3$	bs 4	bs 16	bs 64	bs 128
Fixed $K=6$	1.01	1.01	0.86	0.89
Adaptive bandit (online)	0.93	0.94	0.98	0.99

Table 8: Per-cell view. Cell format ‘ $K=6$ lift over $K=3$ | bandit % of best-fixed’. $K=6$ helps at low batch on math/code and hurts at high batch by 20–29% on math/MCQ; the bandit stays close to whichever fixed is best. Mixes: *Math (mixed difficulty)* draws 120/96/20/20 from GSM8K/MATH500/AIME 2024/2025; *Cross-domain* draws 96/80/80 from GSM8K/MBPP/MMLU-Redux.

Workload	bs 4	bs 16	bs 64	bs 128
GSM8K	+2% 90%	+3% 91%	−23% 96%	−21% 102%
MATH500	+3% 89%	+4% 89%	−22% 96%	−23% 98%
AIME 2024	−5% 90%	−3% 91%	−1% 96%	−2% 96%
AIME 2025	+1% 92%	+1% 93%	+1% 97%	−0% 98%
HumanEval	+9% 86%	+5% 92%	−4% 104%	+35% 74%
MBPP	−0% 96%	+0% 96%	−8% 102%	−4% 94%
MMLU-Pro	−1% 92%	+2% 93%	−21% 96%	−20% 99%
MMLU-Redux	−4% 95%	−7% 93%	−27% 97%	−29% 98%
<i>Math (mixed difficulty)</i>	+2% 89%	+4% 89%	−21% 96%	−22% 98%
<i>Cross-domain mix</i>	+1% 93%	+0% 92%	−17% 96%	−20% 101%

the $K=3$ chain we need to compare per-head profiles; (ii) EAGLE-3 [Li et al., 2025b] uses a single recurrent draft module, not per-depth heads, so per-head acceptance is not defined in the same sense; (iii) the largest open MTP-family checkpoints are attached to backbones an order of magnitude larger than our target, so any MAL comparison would confound drafter quality with base-model quality. MiMo-7B-RL releases all three chained heads on an RL-tuned backbone at $\approx 7B$ parameters, which is the smallest gap to the natural target size for a $\approx 200M$ -per-head chain of ours.

We choose Qwen3-8B [Yang and Team, 2025] as the target because it is the closest freely available base model to MiMo-7B in parameter count (within $\approx 15\%$) that reports competitive quality on the four math benchmarks used in Table 1. A 70B-class target would change every downstream number and confound the training-recipe question we ask: with the same base-model quality budget,

Table 9: K=6 checkpoint per-head acceptance (H_1-H_6 , %) and MAL at batch 4 across ten workloads (arm fixed_k6 of the bandit sweep).

Workload	MAL	$H_1/H_2/H_3/H_4/H_5/H_6$ (%)
GSM8K	3.91	91/78/65/31/18/10
MATH500	4.05	92/80/68/35/21/13
AIME 2024	3.60	89/72/56/24/13/7
AIME 2025	3.68	89/73/57/27/14/8
<i>Math (mixed difficulty)</i>	3.94	91/78/64/32/18/10
HumanEval	3.90	90/74/58/36/20/12
MBPP	3.50	85/68/50/30/14/8
MMLU-Redux	2.96	80/57/37/14/6/2
MMLU-Pro	3.56	86/68/52/26/15/9
<i>Cross-domain mix</i>	3.38	85/66/48/22/11/6

how close to joint pretraining does a plain post-training pass on the chained heads get? Matching parameter counts to within $\approx 15\%$ makes the answer readable off the MAL numbers.

F Reproducibility details

This section collects the setup details needed to reproduce our numbers. Code and weights are not released with this submission; the recipe below is complete enough for independent re-implementation.

Head architecture. Each head is one transformer decoder layer at the backbone’s width (4096 hidden, 32 attention heads, 14336 FFN, RMSNorm, SwiGLU, RoPE, matched to Qwen3-8B), followed by the frozen backbone LM head. Head k receives $[h_t; \text{embed}(\arg \max_v p^{(k-1)}(v \mid h_t, \dots))]$ as input; the concatenation is projected to hidden width by a linear layer before the transformer layer. Per-head parameter count is $\approx 200\text{M}$ ($\approx$ one Qwen3-8B decoder layer), matched to MiMo-7B.

Training. 5 epochs on $\approx 2.5\text{B}$ target-generated tokens per epoch ($\approx 19\text{k}$ optimiser steps at effective batch 240). AdamW with $\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.1, learning rate 5×10^{-5} , cosine schedule to zero with 3% linear warm-up. Gradient checkpointing on the frozen backbone (activations recomputed on the fly, no backbone parameters updated). Sequence length 2048, packed. The $K=6$ variant uses the same schedule with two additional heads sharing per-head architecture.

On-policy corpus. 1.89M reasoning prompts from AM-Thinking [a-m-team, 2024]. Each prompt is completed by the frozen Qwen3-8B in thinking mode with the deployment decoding parameters (temperature 0.6, top- p 0.95, top- k 20). Prompt/response pairs are concatenated verbatim; no filtering.

Ablation architectures. Each ablation replaces one component of the base recipe and is trained on the raw AM-Thinking corpus (Row 2 of Table 4) with the same schedule and optimiser as the base recipe.

GRU cross-head gate. At each head boundary $k-1 \rightarrow k$ we insert a single-layer GRU cell that consumes the current head’s hidden state $h_t^{(k-1)} \in \mathbb{R}^{4096}$ and the argmax-token embedding of head $k-1$ (dimension 4096, from the backbone embedding table), and emits a gated hidden state $\tilde{h}_t^{(k)} \in \mathbb{R}^{4096}$ that is consumed by head k in place of the base h_t . Concretely, letting $g^{(k)} \in \mathbb{R}^{4096}$ be the GRU state, $g^{(k)} = \text{GRU}(g^{(k-1)}, [h_t^{(k-1)}; \text{embed}(\hat{x}_{t+k-1}^{(k-1)})])$ and $\tilde{h}_t^{(k)} = W_o g^{(k)} + h_t^{(k-1)}$ with $W_o \in \mathbb{R}^{4096 \times 4096}$ and a residual connection to preserve the base signal. Gate parameters are $\approx 50\text{M}$ per handoff, trainable end-to-end; backbone frozen.

Mamba-style cross-head gate. Same interface as the GRU gate but with a selective-state-space block [Gu and Dao, 2023] replacing the GRU cell. The block operates on a length- K virtual sequence obtained by stacking $[h_t^{(0)}, \dots, h_t^{(k-1)}]$ across head indices at position t , with $d_{\text{model}} = 4096$, state dimension $d_{\text{state}} = 16$, expansion factor 2, and a length-4 convolution. Because our chain has only $K=3$ or $K=6$ steps, the selective scan reduces to a small unrolled recursion; we implement it as the

reference formulation in Gu and Dao [2023] without their kernel-fused variant, since the sequence length does not warrant the specialised CUDA path. The residual and output projection match the GRU gate. Gate parameters are $\approx 80\text{M}$ per handoff.

Shallow LoRA. LoRA adapters [Hu et al., 2021] of rank $r=8$, $\alpha=16$, dropout 0.05, on the Q/K/V/O attention projections of the last 4 of the 36 backbone layers. Backbone learning rate 1×10^{-5} (heads at 5×10^{-5}). Adapter parameter count $\approx 1\text{M}$ total.

Deep LoRA. Same rank/ α /dropout/backbone LR as shallow, applied to the Q/K/V/O attention projections of the last 18 of the 36 backbone layers (i.e. half the backbone). Adapter parameter count $\approx 4.7\text{M}$ total.

Full-vocabulary KL. Loss $D_{\text{KL}}(p_\theta \| p^{(k)})$ over the full vocabulary at every position, replacing per-token hard cross-entropy in Eq. ??.

Verification. vLLM V1 speculative decoding path with our verification-rule extensions in §3. Chain threshold in Eq. 1 enforced in log-space with a clamp $\log(p) \geq -27.6$ (i.e. $p \geq 10^{-12}$).

Adaptive- K controller (LCB). The controller is an online multi-armed bandit over six arms $\{K=1, \dots, K=6\}$; Each arm’s reward is the observed batched throughput $\text{tps}_a = \text{accepted tokens/wall-clock}$ over a *measurement window* of $W=8$ steps at near-full occupancy (a step counts only if the in-flight batch is $\geq 3/4$ of capacity, so wave-drain tails are dropped). After $\geq n_{\min}=2$ round-robin windows per arm, per-arm running estimates $(\hat{\mu}_a, \hat{v}_a)$ update by exponential smoothing with $\beta=0.7$:

$$\hat{\mu}_a \leftarrow \beta \hat{\mu}_a + (1-\beta) \text{tps}_a, \quad \hat{v}_a \leftarrow \beta \hat{v}_a + (1-\beta) (\text{tps}_a - \hat{\mu}_a^{\text{old}})^2,$$

so $\hat{s}_a = \sqrt{\hat{v}_a}$ is the arm’s measurement spread. The controller then exploits $\arg \max_a \hat{\mu}_a$ with periodic re-exploration every $P=8$ windows. *Commit* fires the first time the argmax arm has been the argmax for $C=3$ consecutive post-settle windows *and* leads the runner-up by a relative margin $\delta=0.015$ (i.e. $(\hat{\mu}_{(1)} - \hat{\mu}_{(2)})/\hat{\mu}_{(2)} \geq \delta$); if neither condition is met within $W_{\max}=30$ windows a commit is forced. At commit we select the arm with the highest lower-confidence bound $\hat{\mu}_a - z\hat{s}_a$ with $z=1$, rather than the raw argmax. After commit the depth is frozen for the rest of the session: no classifier forward, no window bookkeeping, no per-step host sync. Reported numbers correspond to the post-commit segment. A warm-up of 100 steps precedes the first window to let the KV cache and CUDA-graph pool reach steady state.

Algorithm 1 Adaptive- K throughput bandit (LCB, one serving session).

```

1: Input: arms  $\mathcal{A} = \{K=1, \dots, 6\}$ ; window  $W=8$ ; settle  $n_{\min}=2$ ; explore period  $P=8$ ; commit
   stability  $C=3$ ; margin  $\delta=0.015$ ; hard cap  $W_{\max}=30$ ; LCB  $z=1$ ; EMA  $\beta=0.7$ ; occupancy  $\geq 3/4$ 
2:  $(\hat{\mu}_a, \hat{v}_a, N_a) \leftarrow (0, 0, 0)$  for  $a \in \mathcal{A}$ ;  $\text{best}_{\text{prev}} \leftarrow -1$ ;  $\text{run} \leftarrow 0$ ;  $\text{committed} \leftarrow -1$ ;  $w \leftarrow 0$ 
3: while decoding do
4:   collect a window of  $W$  near-full-occupancy steps under the current arm  $a$ ; measure  $\text{tps}_a$ 
5:    $(\hat{\mu}_a, \hat{v}_a) \leftarrow (\beta \hat{\mu}_a + (1-\beta)\text{tps}_a, \beta \hat{v}_a + (1-\beta)(\text{tps}_a - \hat{\mu}_a^{\text{old}})^2)$ ;  $N_a += 1$ ;  $w += 1$ 
6:   if  $\min_a N_a < n_{\min}$  then  $\triangleright$  settle: round-robin to the least-sampled arm
7:      $a \leftarrow \arg \min_a N_a$ ; continue
8:   end if
9:    $\text{best} \leftarrow \arg \max_a \hat{\mu}_a$ ;  $\text{run} \leftarrow \text{run} + 1$  if  $\text{best} = \text{best}_{\text{prev}}$  else 1;  $\text{best}_{\text{prev}} \leftarrow \text{best}$ 
10:   $\text{lead} \leftarrow (\hat{\mu}_{(1)} - \hat{\mu}_{(2)})/\hat{\mu}_{(2)}$ 
11:  if  $\text{run} \geq C$  and  $\text{lead} \geq \delta$ , or  $w \geq W_{\max}$  then
12:     $\text{committed} \leftarrow \arg \max_a (\hat{\mu}_a - z\sqrt{\hat{v}_a})$   $\triangleright$  LCB tie-break: reliably-good beats spiky
13:    return committed  $\triangleright$  freeze depth for the rest of the session
14:  end if
15:   $a \leftarrow \text{best}$  if  $w \bmod P \neq 0$  else next arm  $\neq \text{best}$   $\triangleright$  periodic re-exploration
16: end while

```

Evaluation. We use the vLLM [Leviathan et al., 2023] implementation of speculative decoding on Qwen3-8B for both our checkpoint and MiMo-7B-RL. Batch 4, single A100-80GB GPU, thinking mode with the training-time decoding parameters. Math final answers extracted from `\boxed{\cdot}` and scored by `math_verify`; MCQ by single-letter match; code by unit-test execution; CNN/DailyMail [Nallapati et al., 2016] by ROUGE-1/2/L [Lin, 2004] against reference highlights.

Compute. Head training on 4 nodes of $8\times A100$ -80GB with tensor parallel 1, data parallel 32. One epoch on 2.5B tokens takes ≈ 8 h; 5 epochs ≈ 40 h. The $K=6$ variant costs $\approx 1.4\times$ the $K=3$ variant.

LLM use disclosure

Large language models were used for editing prose in this manuscript and for scaffolding experimental scripts and figure code. All numerical results, tables, figures, and technical claims were checked by the authors against source-code outputs; the LLM was not used to generate any of the reported numbers or to author any technical claim.